

OpenCode → oh-my-opencode 完整调用流程分析

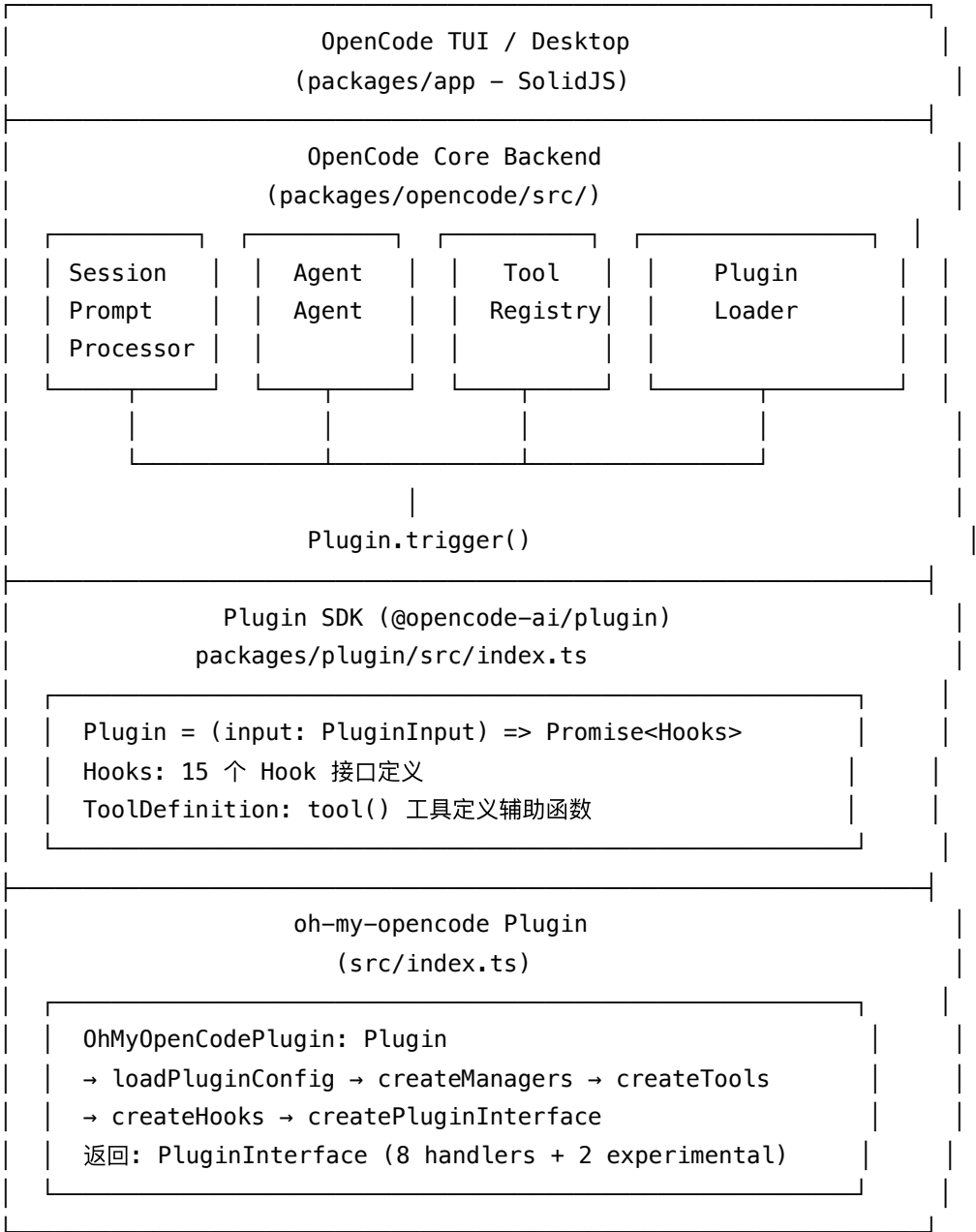
基于 OpenCode 源码 (<https://github.com/anomalyco/opencode>) 与 oh-my-opencode 源码的交叉分析

目录

1. 架构总览
2. 插件 SDK 类型系统
3. 插件加载流程
4. 8 个 Hook 调用链详解
5. Tool 注册与执行流程
6. Agent 注册流程
7. 完整生命周期时序图
8. 数据流向汇总

1. 架构总览

OpenCode 采用 **插件宿主架构 (Plugin Host Architecture)**，核心由三层组成：



关键目录对照：

角色	OpenCode 路径	oh-my-opencode 路径
插件 SDK 类型	packages/plugin/src/index.ts	@opencode-ai/plugin (npm 依赖)
插件加载器	packages/opencode/src/plugin/index.ts	—
会话处理器	packages/opencode/src/session/prompt.ts	—
流处理器	packages/opencode/src/session/processor.ts	—
LLM 调用	packages/opencode/src/session/llm.ts	—

角色	OpenCode 路径	oh-my-opencode 路径
Agent 定义	packages/opencode/src/agent/agent.ts	—
Tool 注册表	packages/opencode/src/tool/registry.ts	—
插件入口	—	src/index.ts
插件接口	—	src/plugin-interface.ts
Config Handler	—	src/plugin-handlers/config-handler.ts

2. 插件 SDK 类型系统

2.1 PluginInput (宿主传入)

```
// packages/plugin/src/index.ts
export type PluginInput = {
  client: ReturnType<typeof createOpencodeClient> // SDK 客户端
  project: Project // 项目信息
  directory: string // 项目目录
  worktree: string // Git worktree 根
  serverUrl: URL // 后端服务 URL
  $: BunShell // Bun Shell ($`cmd`)
}
```

2.2 Plugin 类型 (插件导出)

```
export type Plugin = (input: PluginInput) => Promise<Hooks>
```

2.3 Hooks 接口 (15 个 Hook)

```
export interface Hooks {  
  // === 核心 Hooks ===  
  config?: (input: Config) => Promise<void>  
  event?: (input: { event: Event }) => Promise<void>  
  tool?: { [key: string]: ToolDefinition }  
  auth?: AuthHook  
  
  // === Chat 阶段 Hooks ===  
  "chat.message"?: (input, output) => Promise<void>  
  "chat.params"?: (input, output) => Promise<void>  
  "chat.headers"?: (input, output) => Promise<void>  
  
  // === 权限 Hook ===  
  "permission.ask"?: (input, output) => Promise<void>  
  
  // === 命令 Hook ===  
  "command.execute.before"?: (input, output) => Promise<void>  
  
  // === Tool 执行 Hooks ===  
  "tool.execute.before"?: (input, output) => Promise<void>  
  "tool.execute.after"?: (input, output) => Promise<void>  
  "shell.env"?: (input, output) => Promise<void>  
  
  // === 实验性 Hooks ===  
  "experimental.chat.messages.transform"?: (input, output) => Promise<void>  
  "experimental.chat.system.transform"?: (input, output) => Promise<void>  
  "experimental.session.compacting"?: (input, output) => Promise<void>  
  "experimental.text.complete"?: (input, output) => Promise<void>  
  
  // === Tool 定义 Hook ===  
  "tool.definition"?: (input, output) => Promise<void>  
}
```

2.4 ToolDefinition (工具定义)

```
// packages/plugin/src/tool.ts
export type ToolContext = {
  sessionId: string
  messageId: string
  agent: string
  directory: string           // 项目目录
  worktree: string           // Worktree 根
  abort: AbortSignal
  metadata(input: { title?: string; metadata?: string }): void
  ask(input: AskInput): Promise<void>
}

export function tool<Args extends z.ZodRawShape>(input: {
  description: string
  args: Args
  execute(args, context: ToolContext): Promise<string>
}) { return input }

export type ToolDefinition = ReturnType<typeof tool>
```

3. 插件加载流程

3.1 OpenCode 侧: Plugin.init()

源文件: packages/opencode/src/plugin/index.ts

```

Plugin.init()
└─ state() [Instance.state 懒加载单例]
  └─ 创建 PluginInput { client, project, directory, worktree, serverUrl, $ }
    └─ [Phase 1] 加载内置插件 (INTERNAL_PLUGINS)
      └─ CodexAuthPlugin    (./codex.ts)
      └─ CopilotAuthPlugin (./copilot.ts)
      └─ GitlabAuthPlugin   (@gitlab/opencode-gitlab-auth)
      └─ 每个: await plugin(input) → hooks.push(result)
    └─ [Phase 2] 加载 npm 插件 (config.plugin + BUILTIN)
      └─ BUILTIN = ["opencode-anthropic-auth@0.0.13"]
      └─ 从 config.plugin 读取用户配置的插件列表
      └─ 合并: [...BUILTIN, ...config.plugin]
      └─ 对每个插件:
        └─ BunProc.install(pkg, version) // npm 安装
        └─ import(plugin)                // 动态导入
        └─ Object.entries(mod) 遍历所有导出
          └─ 去重 (seen Set 防止 default + named 重复)
            └─ await fn(input) → hooks.push(result)
          └─ 错误处理 → Bus.publish(Session.Event.Error)
      └─ return { hooks: Hooks[], input: PluginInput }
    └─ 对每个 hook 调用 hook.config?.(config) // 触发 config hook
  └─ Bus.subscribeAll → 转发所有事件到 hook["event"]

```

关键代码片段:

```

// packages/opencode/src/plugin/index.ts
const state = Instance.state(async () => {
  const input: PluginInput = {
    client,
    project: Instance.project,
    worktree: Instance.worktree,
    directory: Instance.directory,
    serverUrl: Server.url(),
    $: Bun.$,
  }

  // 加载插件
  for (let plugin of plugins) {
    await import(plugin).then(async (mod) => {
      const seen = new Set()
      for (const [_name, fn] of Object.entries(mod)) {
        if (seen.has(fn)) continue
        seen.add(fn)
        hooks.push(await fn(input)) // ← 调用 oh-my-opencode 的导出函数
      }
    })
  }
  return { hooks, input }
})

```

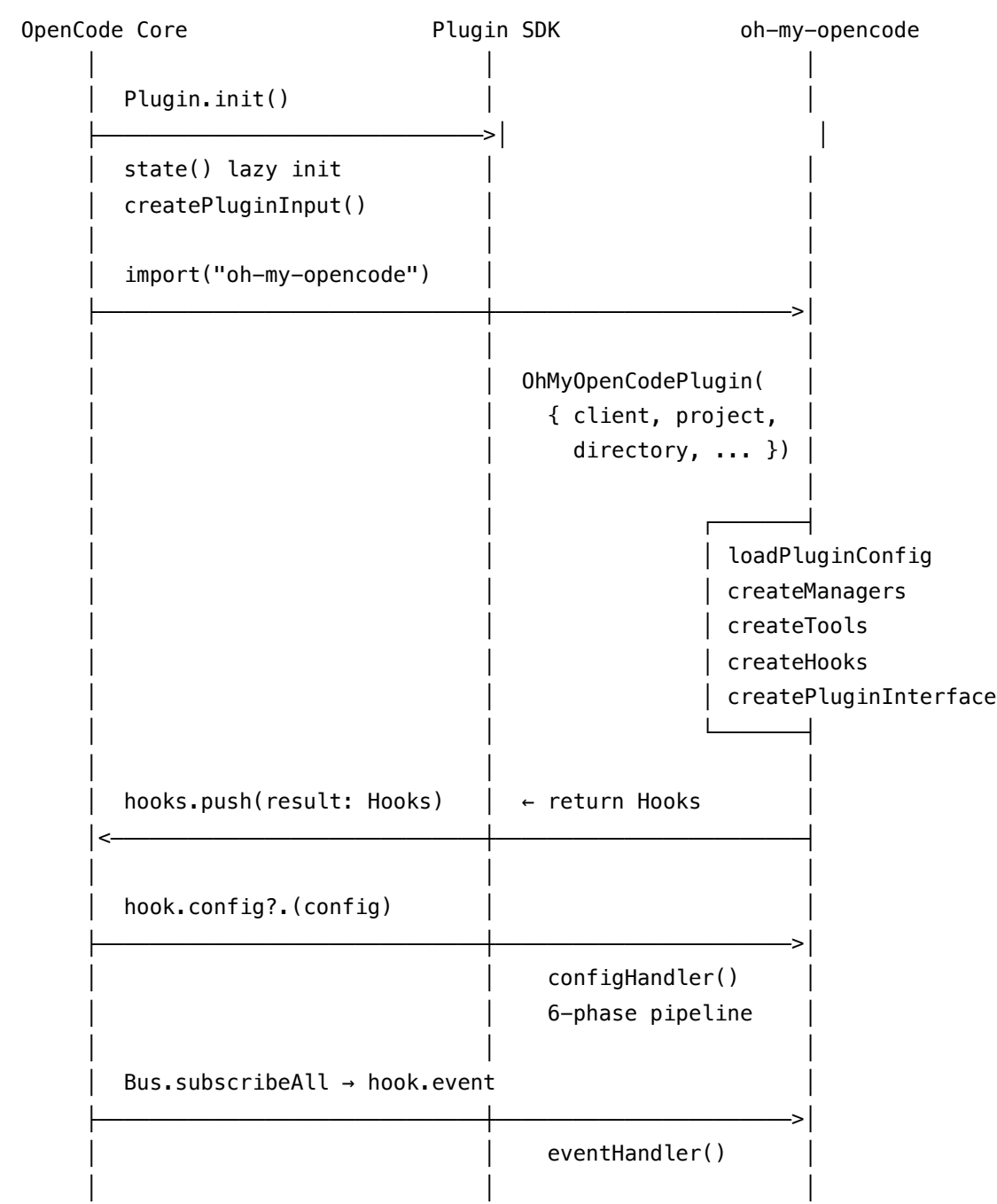
3.2 oh-my-opencode 侧: OhMyOpenCodePlugin(input)

源文件: src/index.ts

OhMyOpenCodePlugin(ctx: PluginInput)

- |
 - |— injectServerAuthIntoClient(ctx.client) // 注入认证
 - |— startTmuxCheck() // 异步检测 tmux
- |
 - |— loadPluginConfig(ctx.directory, ctx) // JSONC 多层配置
 - |— 项目级: .opencode/oh-my-opencode.jsonc
 - |— 用户级: ~/.config/opencode/oh-my-opencode.jsonc
 - |— Zod v4 校验 + 迁移
- |
 - |— createManagers()
 - |— TmuxSessionManager
 - |— BackgroundManager (5 并发/模型)
 - |— SkillMcpManager
 - |— configHandler (6 阶段配置管道)
- |
 - |— createTools()
 - |— 构建 SkillContext
 - |— 构建 AvailableCategories
 - |— 注册 26 个工具到 ToolsRecord
 - |— 按 disabled_tools 过滤
- |
 - |— createHooks()
 - |— Core Hooks (37)
 - |— Continuation Hooks (7)
 - |— Skill Hooks (2)
- |
 - |— createPluginInterface()
 - |— 返回 PluginInterface (8 handlers)
 - |— tool: ToolsRecord
 - |— config: configHandler
 - |— chat.message → createChatMessageHandler
 - |— chat.params → createChatParamsHandler
 - |— chat.headers → createChatHeadersHandler
 - |— event → createEventHandler
 - |— tool.execute.before → createToolExecuteBeforeHandler
 - |— tool.execute.after → createToolExecuteAfterHandler
 - |— experimental.chat.messages.transform → createMessagesTransformHandler
 - |— experimental.chat.system.transform → createSystemTransformHandler
 - |— experimental.session.compacting (直接在 index.ts 中定义)

3.3 加载时序图



4. 8 个 Hook 调用链详解

4.1 config Hook

触发时机: `Plugin.init()` 阶段，所有插件加载完成后

OpenCode 调用代码 (`packages/opencode/src/plugin/index.ts`):

```
export async function init() {
  const hooks = await state().then((x) => x.hooks)
  const config = await Config.get()
  for (const hook of hooks) {
    await hook.config?.(config) // ← 传入完整 OpenCode 配置对象
  }
}
```

oh-my-opencode 接收 (src/plugin-handlers/config-handler.ts):

```
export function createConfigHandler(deps) {
  return async (config: Record<string, unknown>) => {
    // Phase 1: Provider 配置 (模型缓存)
    applyProviderConfig({ config, modelCacheState })

    // Phase 2: 加载插件组件 (skills, commands)
    const pluginComponents = await loadPluginComponents({ pluginConfig })

    // Phase 3: Agent 配置 (11 agents → config.agent)
    const agentResult = await applyAgentConfig({...})

    // Phase 4: Tool 配置 (26 tools → 已通过 hook.tool 注册)
    applyToolConfig({ config, pluginConfig, agentResult })

    // Phase 5: MCP 配置 → config.mcp
    await applyMcpConfig({...})

    // Phase 6: Command 配置 → config.command
    await applyCommandConfig({...})
  }
}
```

数据流:

OpenCode Config 对象 (可变引用)

— config.agent = { sisyphus: {...}, hephaestus: {...}, ... }	← applyAgentConfig 写入
— config.mcp = { websearch: {...}, context7: {...}, ... }	← applyMcpConfig 写入
— config.command = { review: {...}, plan: {...}, ... }	← applyCommandConfig 写入
— config.formatter (保留原值)	

4.2 event Hook

触发时机: 所有 Bus 事件 (session.created, session.deleted, session.error, etc.)

OpenCode 调用代码 (packages/opencode/src/plugin/index.ts):

```
Bus.subscribeAll(async (input) => {
  const hooks = await state().then((x) => x.hooks)
  for (const hook of hooks) {
    hook["event"]?.({ event: input })
  }
})
```

oh-my-opencode 接收 (src/plugin/event.ts):

```
event({ event })
|
| — event.type === "session.created"
|   | — firstMessageVariantGate.markSessionCreated()
|   | — hooks.sessionSetup?.({ sessionInfo })
|
| — event.type === "session.deleted"
|   | — firstMessageVariantGate.clear()
|   | — backgroundManager.cleanup()
|   | — hooks.sessionDeleted?.()
|
| — event.type === "session.error"
|   | — hooks.errorHandler?.()
|
| — 其他事件 → 转发到相应 hook
```

4.3 chat.message Hook

触发时机: 用户发送消息时，在消息保存到数据库之前

OpenCode 调用代码 (packages/opencode/src/session/prompt.ts → createUserMessage()):

```

await Plugin.trigger(
  "chat.message",
  {
    sessionID: input.sessionID,
    agent: input.agent,
    model: input.model,
    messageID: input.messageID,
    variant: input.variant,
  },
  {
    message: info,      // UserMessage (可修改)
    parts: parts,      // Part[] (可修改)
  },
)
// 然后保存: Session.updateMessage(info) + Session.updatePart(parts)

```

oh-my-opencode 接收 (src/plugin/chat-message.ts):

```

chat.message(input, output)
├─ 首条消息变体 (firstMessageVariantGate)
│   └─ 对首条消息应用 variant override
├─ hooks.sessionSetup
│   └─ 会话初始化逻辑 (keywords 检测等)
├─ hooks.keywordDetector
│   └─ 检测提示词中的关键词 (如 "plan", "build")
└─ hooks.messageTransformer
    └─ 消息内容变换

```

4.4 chat.params Hook

触发时机: LLM 流式请求发起前, 组装参数时

OpenCode 调用代码 (packages/opencode/src/session/llm.ts → stream()):

```
const params = await Plugin.trigger(
  "chat.params",
  {
    sessionId: input.sessionID,
    agent: input.agent,
    model: input.model,
    provider, // ProviderContext
    message: input.user, // UserMessage
  },
  {
    temperature: ..., // 可修改
    topP: ..., // 可修改
    topK: ..., // 可修改
    options: {}, // 可修改 (provider options)
  },
)
// 然后使用 params.temperature, params.topP 等
```

oh-my-opencode 接收 (src/plugin/chat-params.ts):

```
chat.params(input, output)
└─ hooks.anthropicEffort
   └─ 调整 Anthropic 模型的 effort level
       output.options.thinking.budget_tokens = ...
```

4.5 chat.headers Hook

触发时机: LLM 请求发起前, 设置 HTTP headers

OpenCode 调用代码 (packages/opencode/src/session/llm.ts → stream()):

```
const { headers } = await Plugin.trigger(
  "chat.headers",
  {
    sessionId: input.sessionID,
    agent: input.agent,
    model: input.model,
    provider,
    message: input.user,
  },
  { headers: {} },
)
// headers 合并到 streamText() 的 headers 参数
```

oh-my-opencode 接收 (src/plugin/chat-headers.ts):

```
chat.headers(input, output)
└─ 注入自定义 headers (如认证 token)
```

4.6 tool.execute.before Hook

触发时机: 工具执行前, 在 Tool 参数已解析后

OpenCode 调用代码 (packages/opencode/src/session/prompt.ts → resolveTools() 中每个工具的 execute 包装):

```
// 对每个注册的工具, 包装 execute:
async execute(args, options) {
  const ctx = context(args, options)
  await Plugin.trigger(
    "tool.execute.before",
    {
      tool: item.id,          // 工具名
      sessionID: ctx.sessionID,
      callID: ctx.callID,
    },
    { args },                // args 可修改
  )
  const result = await item.execute(args, ctx)
  // ...
}
```

也在 subtask 工具单独调用 (prompt.ts → loop中):

```
await Plugin.trigger(
  "tool.execute.before",
  { tool: "task", sessionID, callID: part.id },
  { args: taskArgs },
)
```

oh-my-opencode 接收 (src/plugin/tool-execute-before.ts):

```
tool.execute.before(input, output)
├─ hooks.fileGuard
│   └─ 文件操作保护（敏感路径检测）
├─ hooks.labelTruncator
│   └─ 截断过长的工具参数标签
└─ hooks.rulesInjector
    └─ 注入自定义规则到 args
```

4.7 tool.execute.after Hook

触发时机: 工具执行完成后

OpenCode 调用代码 (packages/opencode/src/session/prompt.ts → resolveTools() 中工具包装):

```
const result = await item.execute(args, ctx)
const output = { ...result, attachments: ... }
await Plugin.trigger(
  "tool.execute.after",
  {
    tool: item.id,
    sessionID: ctx.sessionID,
    callID: ctx.callID,
    args,
  },
  output, // output 可修改 (title, output, metadata)
)
return output
```

oh-my-opencode 接收 (src/plugin/tool-execute-after.ts):

```
tool.execute.after(input, output)
├─ hooks.outputTruncator
│   └─ 截断过长的工具输出
└─ hooks.metadataStore
    └─ 记录工具执行元数据
```

4.8 experimental.chat.messages.transform Hook

触发时机: 在构建 LLM 消息时，在消息列表发送给 LLM 之前

OpenCode 调用代码 (packages/opencode/src/session/prompt.ts → loop()):

```
await Plugin.trigger(
  "experimental.chat.messages.transform",
  {},
  { messages: msgs },          // MessageV2.WithParts[] (可修改)
)
// 然后: MessageV2.toModelMessages(msgs, model) → 发送给 LLM
```

oh-my-opencode 接收 (src/plugin/messages-transform.ts):

```
experimental.chat.messages.transform(input, output)
├─ hooks.contextInjector
│   └─ 注入上下文信息到消息列表
├─ hooks.thinkingBlockValidator
│   └─ 验证/修复 thinking blocks 格式
└─ hooks.claudeCodeHooks
    └─ Claude Code 兼容性处理
```

4.9 experimental.chat.system.transform Hook

触发时机: 构建 system prompt 时

OpenCode 调用代码 (packages/opencode/src/session/llm.ts → stream()):

```
await Plugin.trigger(
  "experimental.chat.system.transform",
  { sessionID: input.sessionID, model: input.model },
  { system },          // string[] (可修改, 添加/修改 system prompt)
)
```

oh-my-opencode 接收 (src/plugin/system-transform.ts):

```
experimental.chat.system.transform(input, output)
└─ (当前为空实现, 预留扩展点)
```

4.10 experimental.session.compacting Hook

触发时机: 会话压缩 (compaction) 开始前

OpenCode 调用代码: 在 compaction 流程中触发

oh-my-opencode 直接在 index.ts 中定义:

```

"experimental.session.compacting": async (_input, output) => {
  // 1. 保存 todo 状态
  await hooks.compactionTodoPreserver?.capture(_input.sessionID)
  // 2. 转发到 Claude Code 兼容 hooks
  await hooks.claudeCodeHooks?.["experimental.session.compacting"]?._(_input, output)
  // 3. 注入压缩上下文
  if (hooks.compactionContextInjector) {
    output.context.push(hooks.compactionContextInjector(_input.sessionID))
  }
}
}

```

5. Tool 注册与执行流程

5.1 Tool 注册 (OpenCode 侧)

源文件: packages/opencode/src/tool/registry.ts

```

ToolRegistry.state()
|
├─ [Step 1] 加载自定义工具文件
|   └─ Config.directories() → scan "{tool,tools}/*.{js,ts}"
|       └─ fromPlugin(id, def) 转换为 Tool.Info
|
├─ [Step 2] 加载插件工具 ★
|   └─ Plugin.list() → 遍历所有插件的 hooks
|       └─ 对每个 plugin.tool 中的工具:
|           └─ fromPlugin(id, def) → custom.push()
|
└─ all() 汇总所有工具
    └─ 内置工具: Bash, Read, Glob, Grep, Edit, Write, Task, WebFetch, etc.
        └─ custom (来自文件 + 插件)

```

fromPlugin 转换逻辑:

```

function fromPlugin(id: string, def: ToolDefinition): Tool.Info {
  return {
    id,
    init: async (initCtx) => ({
      parameters: z.object(def.args),
      description: def.description,
      execute: async (args, ctx) => {
        const pluginCtx = {
          ...ctx,
          directory: Instance.directory,
          worktree: Instance.worktree,
        } as PluginToolContext
        const result = await def.execute(args, pluginCtx)
        // 自动截断输出
        const out = await Truncate.output(result, {}, initCtx?.agent)
        return { title: "", output: out.content, metadata: { truncated } }
      },
    }),
  }
}

```

5.2 Tool 执行完整链路

用户消息 → LLM 返回 tool_call

```

|
├─ SessionProcessor.process() → stream.fullStream
|   └─ case "tool-call":
|       └─ 记录 toolcalls[callID]
|
└─ ai SDK 自动调用 tool.execute()
    |
    ├─ Plugin.trigger("tool.execute.before", { tool, sessionID, callID }, { args })
    |   └─ oh-my-opencode: fileGuard + labelTruncator + rulesInjector
    |
    ├─ item.execute(args, ctx)
    |   └─ 内置工具: 直接执行 (Bash/Read/Edit/...)
    |       └─ 插件工具: fromPlugin 包装 → def.execute(args, pluginCtx)
    |           └─ oh-my-opencode 的 26 个工具之一执行
    |
    ├─ Plugin.trigger("tool.execute.after", { tool, sessionID, callID, args }, output)
    |   └─ oh-my-opencode: outputTruncator + metadataStore
    |
    └─ return { title, output, metadata, attachments }

```

5.3 oh-my-opencode 的 26 个注册工具

通过 hook.tool 字段返回的工具对象:

工具	功能	执行引擎
task	多 Agent 委派	delegate-task/executor
background_task	后台任务	BackgroundManager
background_status	后台状态查询	BackgroundManager
mcp_websearch	Web 搜索	MCP (Exa/Tavily)
mcp_context7	文档查询	MCP (context7)
mcp_grep_app	代码搜索	MCP (grep.app)
skill_*	Skill 工具	SkillMcpManager
... (共 26 个)		

6. Agent 注册流程

6.1 OpenCode 内置 Agent

源文件: packages/opencode/src/agent/agent.ts

OpenCode 内置定义了以下 agent:

Agent	Mode	描述
build	primary	默认 agent, 完整工具权限
plan	primary	只读分析模式
general	subagent	通用子 agent
explore	subagent	代码搜索子 agent
compaction	primary (hidden)	会话压缩
title	primary (hidden)	标题生成
summary	primary (hidden)	摘要生成

6.2 oh-my-opencode 注入 Agent

通过 config hook 的 Phase 3 (applyAgentConfig), oh-my-opencode 在 OpenCode 的 config.agent 对象上写入 11 个 agent:

```

config.agent = {
  sisyphus: { mode: "primary", model: ..., prompt: ..., ... },
  hephaestus: { mode: "all", model: ..., prompt: ..., ... },
  oracle: { mode: "subagent", model: ..., ... },
  librarian: { mode: "subagent", model: ..., ... },
  explore: { mode: "subagent", model: ..., ... }, // 覆盖内置
  atlas: { mode: "subagent", model: ..., ... },
  prometheus: { mode: "subagent", model: ..., ... },
  metis: { mode: "subagent", model: ..., ... },
  momus: { mode: "subagent", model: ..., ... },
  multimodal: { mode: "subagent", model: ..., ... },
  sisyphus-jr: { mode: "subagent", model: ..., ... },
}

```

OpenCode 如何消费: Agent.state() 中读取 cfg.agent , 合并到内置 agent 列表:

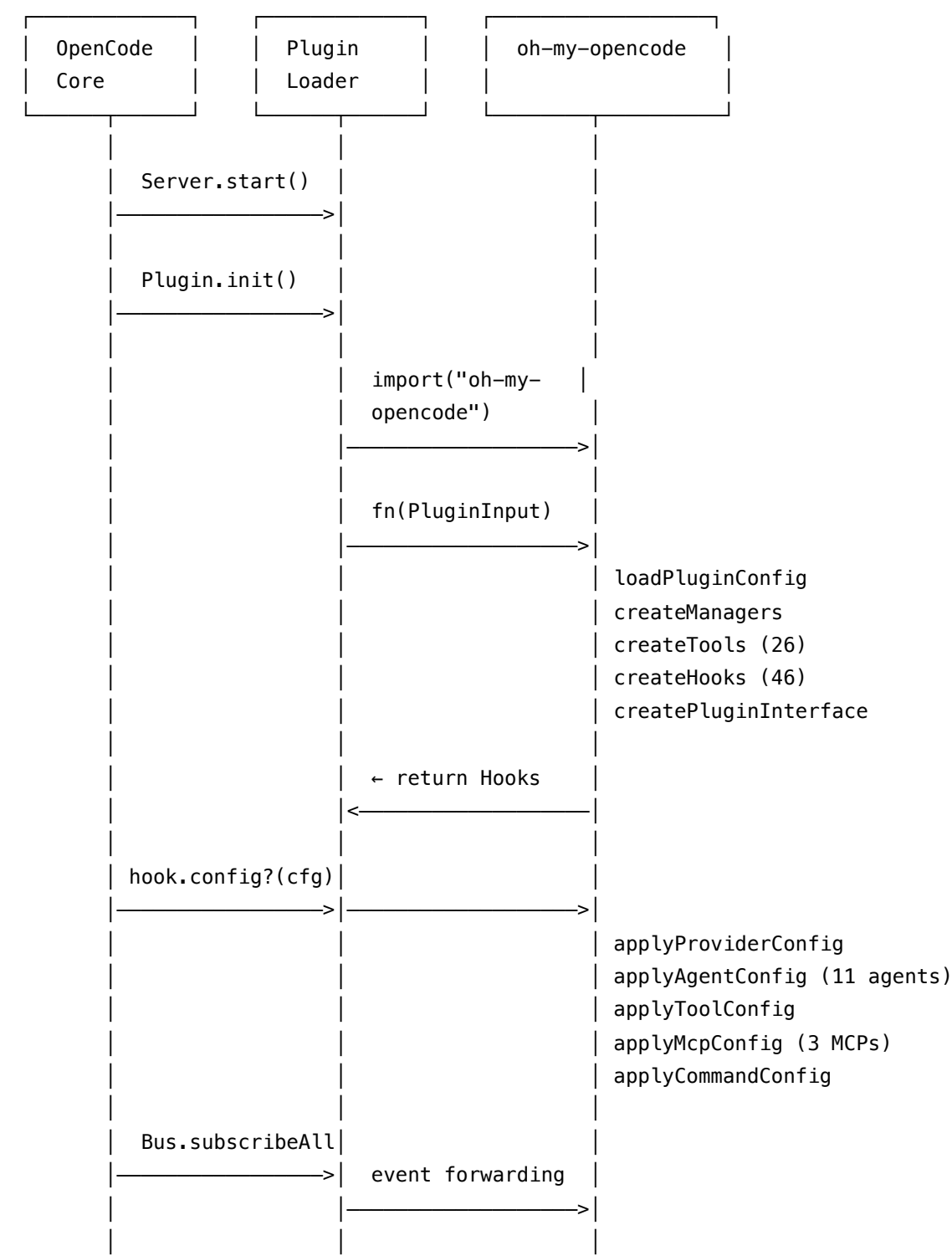
```

// packages/opencode/src/agent/agent.ts
for (const [key, value] of Object.entries(cfg.agent ?? {})) {
  if (value.disable) { delete result[key]; continue }
  let item = result[key]
  if (!item) item = result[key] = { name: key, mode: "all", ... }
  item.model = value.model ? Provider.parseModel(value.model) : item.model
  item.prompt = value.prompt ?? item.prompt
  item.description = value.description ?? item.description
  // ... 合并所有字段
}

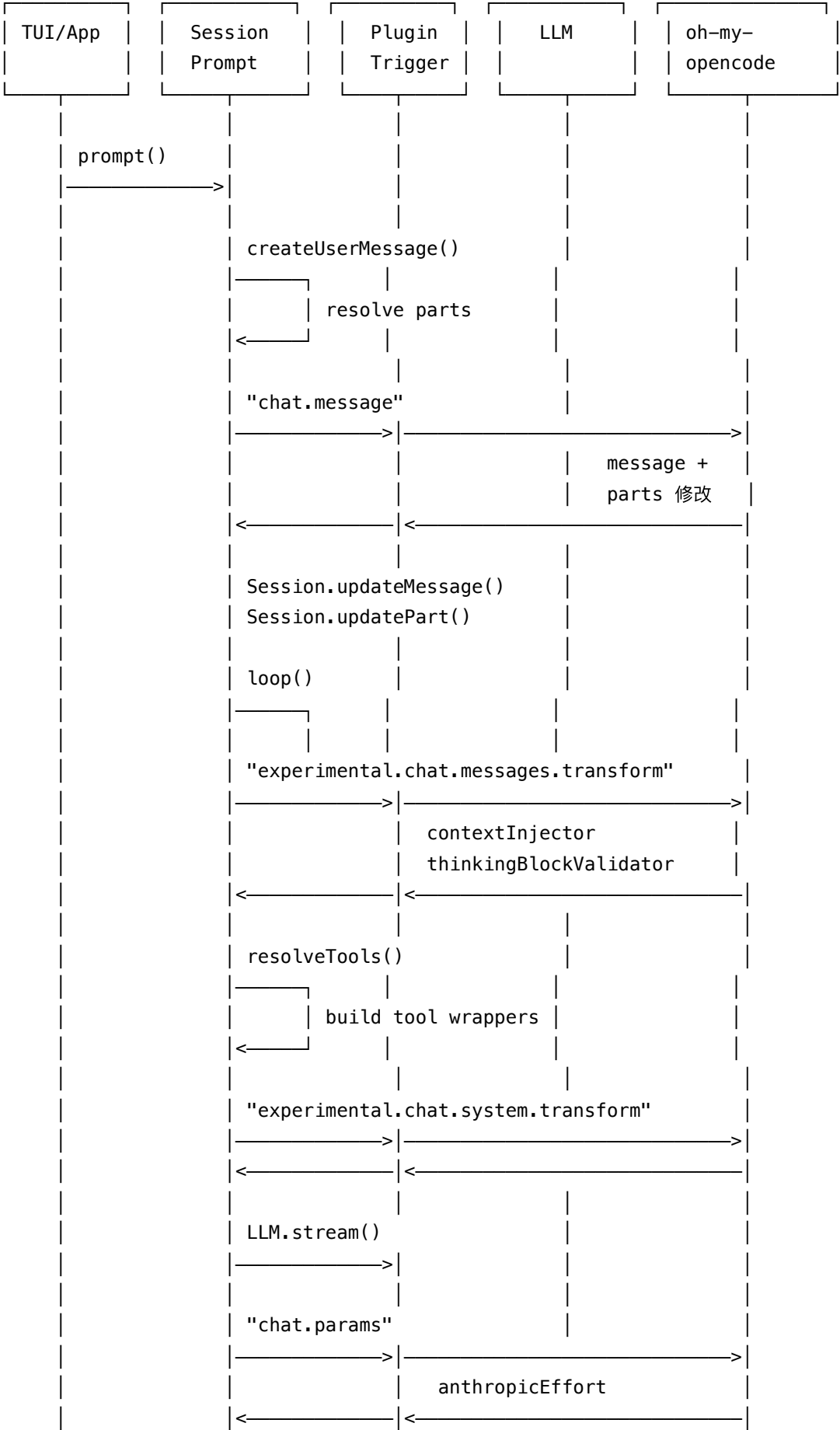
```

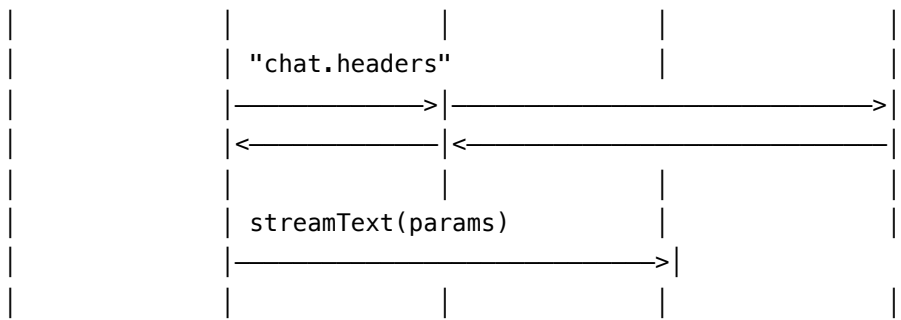
7. 完整生命周期时序图

7.1 启动阶段

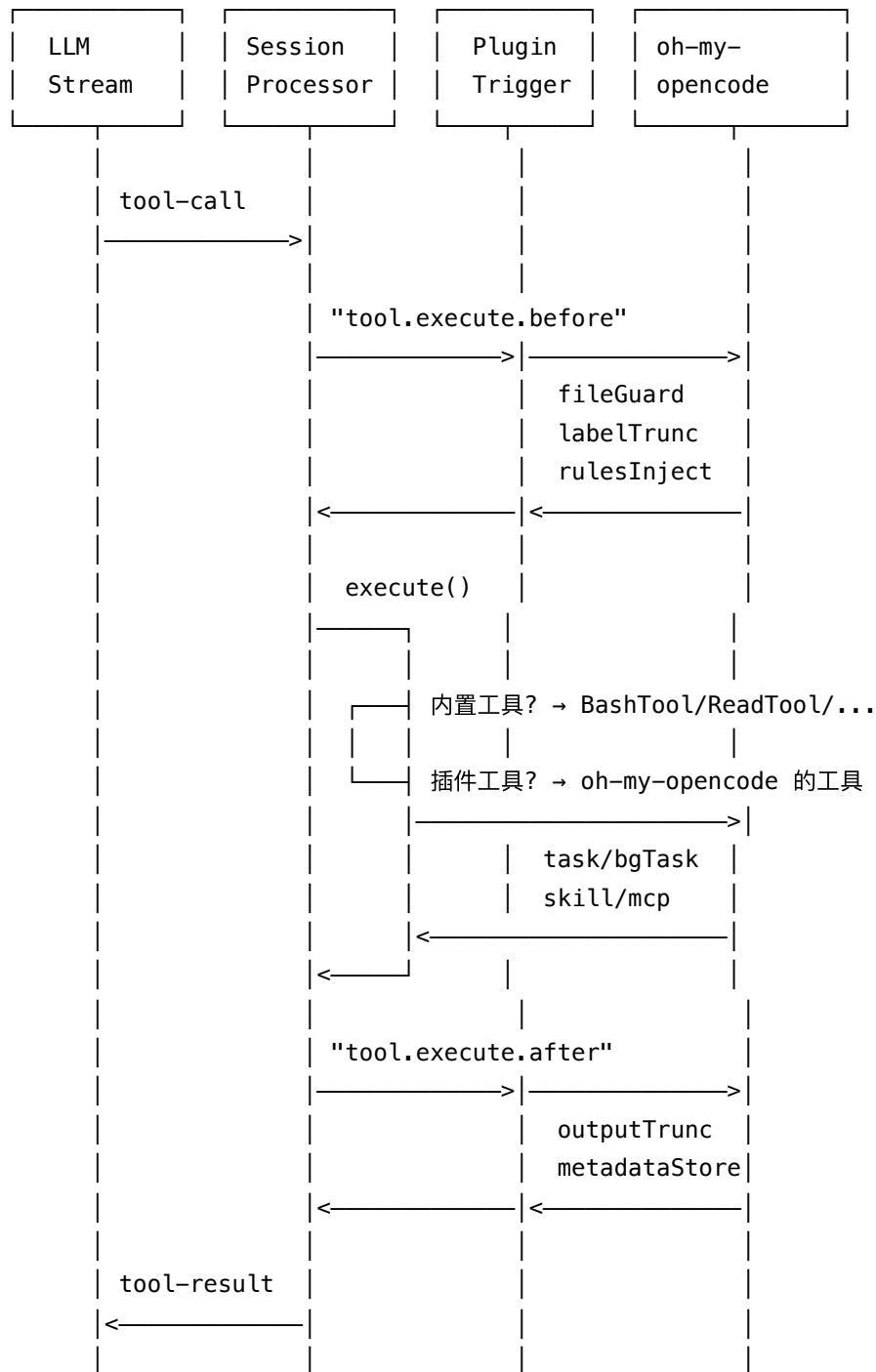


7.2 用户消息处理阶段





7.3 工具执行阶段



8. 数据流向汇总

8.1 Plugin.trigger() 通用模式

OpenCode 中所有 hook 调用遵循同一模式：

```
// packages/opencode/src/plugin/index.ts
export async function trigger<Name>(name, input, output) {
  for (const hook of await state().then((x) => x.hooks)) {
    const fn = hook[name]
    if (!fn) continue
    await fn(input, output)  // ← output 为可变引用，插件直接修改
  }
  return output
}
```

特性:

- **input**: 只读上下文信息（sessionId, agent, model 等）
- **output**: 可变引用，插件直接修改其属性
- **顺序执行**: 多插件按加载顺序依次调用
- **await**: 每个 hook 调用都是 async/await

8.2 Hook 触发点汇总表

Hook	OpenCode 触发位置	触发条件	可修改的数据
config	plugin/index.ts → init()	启动时一次	Config 对象 (agent, mcp, command)
event	plugin/index.ts → Bus.subscribeAll	所有 Bus 事件	— (只读)
chat.message	session/prompt.ts → createUserMessage()	用户发消息	message, parts
chat.params	session/llm.ts → stream()	LLM 调用前	temperature, topP, topK, options
chat.headers	session/llm.ts → stream()	LLM 调用前	headers

Hook	OpenCode 触发位置	触发条件	可修改的数据
tool.execute.before	session/prompt.ts → resolveTools()	工具执行前	args
tool.execute.after	session/prompt.ts → resolveTools()	工具执行后	title, output, metadata
experimental.chat.messages.transform	session/prompt.ts → loop()	每轮循环	messages[]
experimental.chat.system.transform	session/llm.ts → stream()	LLM 调用前	system[]
experimental.session.compacting	compaction 流程	会话压缩时	context[], prompt
experimental.text.complete	session/processor.ts	text 输出完成	text
tool.definition	tool/registry.ts → tools()	工具列表构建	description, parameters
shell.env	session/prompt.ts → shell()	Shell 命令执行	env
command.execute.before	session/prompt.ts → command()	命令执行前	parts
permission.ask	权限系统	权限询问时	status

8.3 oh-my-opencode 使用的 Hook 映射

OpenCode Hook	oh-my-opencode Handler	内部 Hook 数量
config	configHandler (6-phase)	—
event	createEventHandler	5+ hooks
chat.message	createChatMessageHandler	3+ hooks
chat.params	createChatParamsHandler	1 hook (anthropicEffort)
chat.headers	createChatHeadersHandler	1 hook
tool.execute.before	createToolExecuteBeforeHandler	3 hooks
tool.execute.after	createToolExecuteAfterHandler	2 hooks
experimental.chat.messages.transform	createMessagesTransformHandler	3 hooks
experimental.chat.system.transform	createSystemTransformHandler	0 (预留)

OpenCode Hook	oh-my-opencode Handler	内部 Hook 数量
experimental.session.compacting	直接在 index.ts	3 hooks
tool (注册表)	26 工具	—

8.4 Task Tool 跨系统调用链

当 LLM 调用 oh-my-opencode 的 task 工具时的完整链路：

```

LLM → tool_call("task", { prompt, subagent_type: "hephaestus" })
├── [OpenCode] ai SDK → tool.execute()
│   ├── Plugin.trigger("tool.execute.before")
│   │   └── [oh-my-opencode] fileGuard, labelTruncator
│   └── fromPlugin.execute(args, pluginCtx)
│       ├── [oh-my-opencode] task tool
│       │   ├── resolveSubagentExecution()
│       │   │   ├── 验证 agent 存在
│       │   │   └── 解析 model (override → agent.model → fallback)
│       │   ├── 创建子 session (via ctx.client)
│       │   │   ├── [OpenCode] Session.createNext()
│       │   │   │   ├── Bus.publish("session.created")
│       │   │   │   └── Plugin.trigger("event")
│       │   │   └── [oh-my-opencode] sessionSetup
│       │   ├── 发送 prompt (via ctx.client)
│       │   │   ├── [OpenCode] SessionPrompt.prompt()
│       │   │   │   ├── createUserMessage()
│       │   │   │   │   ├── Plugin.trigger("chat.message")
│       │   │   │   │   └── [oh-my-opencode]
│       │   │   └── loop() → LLM.stream()
│       │   │       ├── Plugin.trigger("chat.params")
│       │   │       ├── Plugin.trigger("chat.headers")
│       │   │       ├── Plugin.trigger("experimental.chat.system.transform")
│       │   │       └── streamText() → 子 agent 执行
│       │   │           └── tool_call → ...递归...
│       └── 返回结果
├── Plugin.trigger("tool.execute.after")
│   └── [oh-my-opencode] outputTruncator, metadataStore
└── return { title, output, metadata }
└── LLM 继续推理

```

附录：关键文件索引

OpenCode 侧

文件	职责
packages/plugin/src/index.ts	Plugin SDK 类型定义 (Plugin, Hooks, PluginInput)
packages/plugin/src/tool.ts	ToolDefinition 类型 + tool() 辅助函数
packages/opencode/src/plugin/index.ts	插件加载器 (state, init, trigger, list)
packages/opencode/src/session/prompt.ts	会话主循环 (prompt, loop, command, shell)
packages/opencode/src/session/processor.ts	流处理器 (process, tool-call/result 处理)
packages/opencode/src/session/llm.ts	LLM 调用 (stream, chat.params/headers 触发)
packages/opencode/src/agent/agent.ts	Agent 定义 (内置 build/plan/general/explore)
packages/opencode/src/tool/registry.ts	工具注册表 (内置 + 自定义 + 插件工具)
packages/opencode/src/tool/tool.ts	Tool.Info 类型 + Tool.define() 工具定义

oh-my-opencode 侧

文件	职责
src/index.ts	插件入口 (OhMyOpenCodePlugin)
src/plugin-interface.ts	8 个 Hook handler 组装
src/plugin-handlers/config-handler.ts	config hook: 6 阶段配置管道
src/plugin/chat-message.ts	chat.message handler
src/plugin/chat-params.ts	chat.params handler
src/plugin/chat-headers.ts	chat.headers handler
src/plugin/event.ts	event handler
src/plugin/tool-execute-before.ts	tool.execute.before handler
src/plugin/tool-execute-after.ts	tool.execute.after handler
src/plugin/messages-transform.ts	experimental.chat.messages.transform handler
src/plugin/system-transform.ts	experimental.chat.system.transform handler
src/create-tools.ts	26 工具创建
src/create-hooks.ts	46 hooks 创建 (3 层)

文件	职责
src/create-managers.ts	4 管理器创建